1 Data preprocessing: what features you used, missingness, scaling, PCA choice (if used).

Neither dataset required substantial preprocessing. The heart dataset did not contain any missing values so no imputation was needed, however there were 723 repeated rows in the X_clf and 1023 repeated rows in the y_clf. The repeated rows within the X_clf dataset, and the rows at the corresponding y_clf index were removed. The remaining 300 repeated rows in the y_clf did not need to be removed because they would not impact prediction results since the model is not being trained on those labels. The df_aging dataset contained no repeated rows, but 526 missing values, which were imputed using SimpleImputer(strategy='mean') within a pipeline alongside the model, and StandardScaler() for the training set, and in another pipeline with just StandardScaler() for testing. A pipeline was used to prevent data leakage, although the model was run imputing values on the whole dataset before the train/test split, but there was minimal change in results. Scaling was used for both the heart and aging datasets because scaling greatly reduced the training time for hyper parameter tuning, and to prevent large feature values from dominating the other features when calculating distance between the support vectors and the decision boundary. I used PCA on the aging dataset due to the number of feature columns within the dataset. I reduced to 8 first, because this is the max size allowable with a train/test split of 80/20 (I initially tried doing a higher value but would get an error saying PCA for testing must have n_components between 0 and 8, and needed to use the same number of components for train and test) I tried reducing features to 5 as well but saw little to no change in the results. It seems that overall PCA did not improve performance from the full feature dataset.

2 Modeling hyperparameters: which kernel/params worked best for classification vs regression, and why.

2.1 Heart Dataset

Both RBF and Poly performed similarly well for the heart dataset, however I chose the poly kernel for my model. After hyper parameter tuning my best combination of hyper params was:

```
{
    kernel: 'poly',
    C: 0.001,
    degree: 3,
    gamma: 1
}
```

The heart dataset likely contains features that are not linearly separable explaining why we see better performance with the non-linear kernels. Although many features are linearly separable so we still have good results on the linear kernel. The results for this kernel were:

```
{
    Accuracy: .836066,
    Precision: 0.787878,
    Recall: 0.896551,
    F1: 0.838710
}
```

For all three kernels we get consistently high values, getting 0.78-0.81 across all metrics, however this kernel permutation was outperforming most others achieving accuracy and F1 of 0.83 and a very high recall of 0.89. Precision stays consistent with the other kernels.

2.2 Age Data

For the age dataset we see a similar trend of similar performances between the non-linear poly and rbf kernels, however performance is fairly low across all kernels. There is one RBF result that is marginally better than the rest. The best permutation is:

```
{
```

kernel: 'rbf',
C: 1,
degree: 1,
gamma: 'auto'
}

Again, the features are likely not linearly seperable necessitating a non-linear kernel, which is why the linear kernel has extremely low R2 values relaitve to poy and rbf. For this combination I get these metrics on the full feature set:

{
'MAE': 3.996348,
'RMSE': 5.708456,
'R2': -0.003143
}

We can see poor performance on the full feature set with negative R2 scores and higher MAE and RMSE, indicating we are getting larger errors on our predictions as well as having incredibly poor model explainability based on our R2 value.

3 Metrics visuals: include key scores + confusion matrix/ROC/PR (classification) and parity/residuals (regression).

3.1 Heart SVC

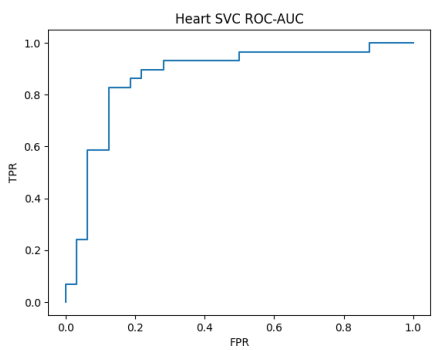

Figure 1: ROC-Auc for Heart SVC

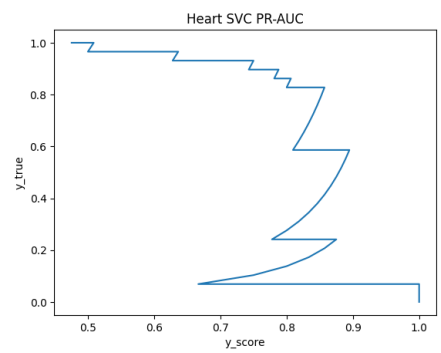

Figure 2: PR-AUC for Heart SVC

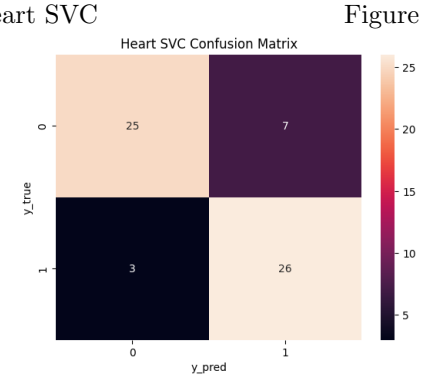

Figure 3: Confusion Matrix for Heart Dataset

As previously stated, we achieved consistently high performance for the heart model across hyperparameter tuning (see figures in Section 4), but our best model produced these results:

{
Accuracy: .836066,
Precision: 0.787878,
Recall: 0.896551,
F1: 0.838710
}

The high accuracy indicates that that model is correctly predicting the positive class (has heart disease) 83% of the time on the test data. We know these positive predictions are in fact correct due to our higher precision, which tells us that the model is correct 78% of the time when predicting the positive class. Recall functions similarly to precision showing that the model is correctly identifying what should and shouldn't be the positive class 89% of the time. The high F1 score shows that the balance between our precision and recall is very good, meaning these two metrics are in agreement with one another 83% of the time. These results can confirmed the the plots in above, where we have high AUC for both ROC and PR showing that ROC and PR meaning these metrics are performing well across various thresholds. We can see the specific breakdown of predictions in the confusion matrix, showing high TP and TN values, and low FP and FN values.

3.2 Age SVR

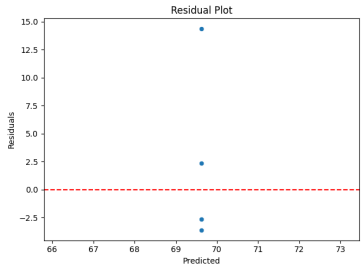

Figure 4: Residuals for the Full Feature Age Model

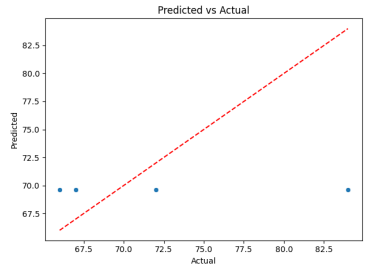

Figure 5: Results for the Full Feature Age Model

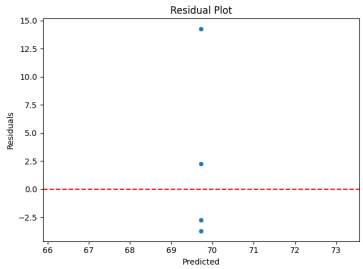

Figure 6: Hyperparameter Sensitivity for RMSE

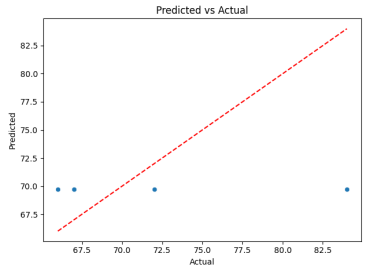

Figure 7: Hyperparameter Sensitivity for RMSE

PCA Components	MAE	RMSE	R2
Full Feature	4.177041	5.670296	0.010224
8 Components	4.237500	5.700384	-0.000308
5 Components	4.237500	5.700384	-0.000308
3 Components	4.237500	5.700384	-0.000308
2 Components	4.237364	5.700379	-0.000306

Looking at the results table we can see feature reduction decreased performance, and generally performance is poor regardless of PCA. This is likely a result of the dataset being incredibly noisy, high dimensional, and having very few data points (rows) to train on. This is evident in the residual and result graphs, given we see all of our data points being incredibly distant from line of best fit, indicating that our performance is not even as good as a random guess. We have high MAE, supported by the tables, showing high errors alongside RMSE, and our R2 values tell us that our model has terrible explainability.

4 Sensitivity: how results changed with scaling/PCA and with C/γ/degree.

4.1 Heart Dataset

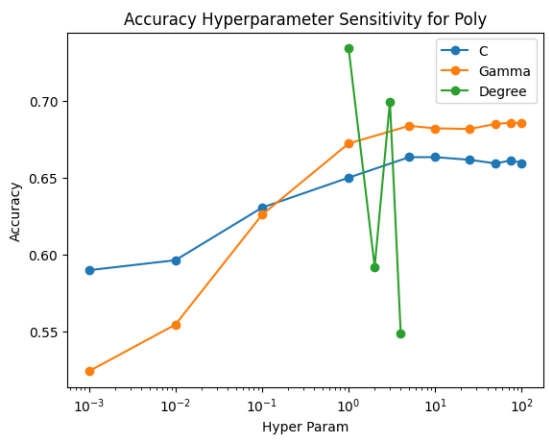

Figure 8: Hyperparameter Sensitivity for Accuracy

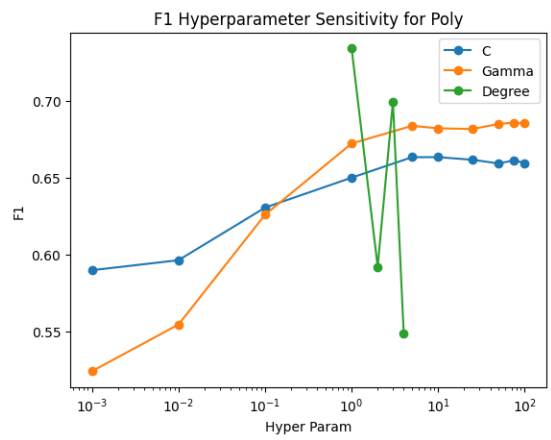

Figure 9: Hyperparameter Sensitivity for F1

I did not get any results without scaling because when I ran these results once the Gamma and Degree got too high it would run infinity (I ran it overnight and I still had not gone through a single iterate of my outermost for loop), but Scaling produced good scores. As we can see the model performance on both F1 and Accuracy the metrics increase as hyperparameters increased, however they did not change drastically after C and Gamma went above 1.0 meaning the model was fairly insensitive to changes within the hyper parameters after a certain point. Although, we can see drastic changes with degree, seeing that degrees of 1 and 3 are ideal to use. The metrics increased as hyperparam value increased, and retained a high accuracy and F1 as Gamma and C increase above 1.0. Note that these graphs aggregate the scores for each hyper parameter to account for each datapoint really being a combination of hyper parameters, as plotting the raw combination results produces difficult to read plots, so the accuracy and F1 scores do not achieve the same high values as in the raw results. The fairly insensitive hyperparameters are likely a result of the smaller dataset having fewer features making the decision boundary easier to learn even for untuned models. For this reason PCA was not used or the heart dataset, since there are not many features, and we have the ability to handle non-linear features with non-linear kernels, feature selection with PCA would not have substantial impact on the results.

4.2 Cancer Dataset

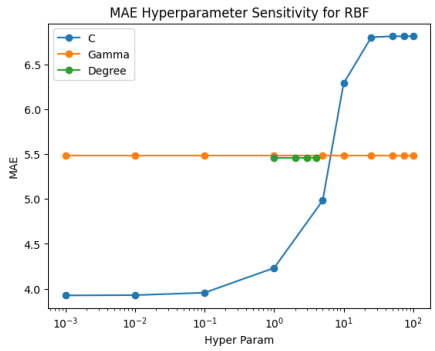

Figure 10: Hyperparameter Sensitivity for MAE

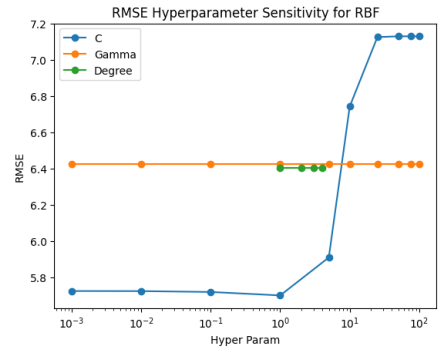

Figure 11: Hyperparameter Sensitivity for RMSE

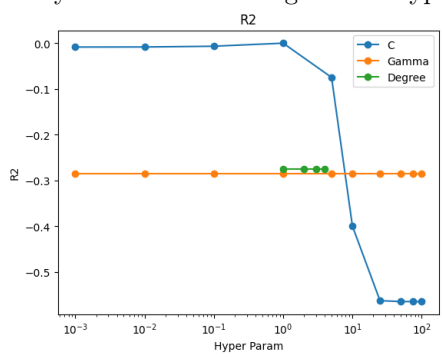

Figure 12: Hyperparameter Sensitivity for RMSE

From the cancer graphs we can see only the C seems to be sensitive, we have almost no change in any of the metrics as Degree and Gamma, however C appears to be somewhat sensitive, with performance across all metrics being better with lower C values, than a sharp decrease in performance once C hits 1 or greater. This poor performance is not entirely unexpected as the dataset has many features, with very few samples per feature (i.e. there are very few rows) so we can expect poor performance as a result of the model not having enough information to learn on, as well as the data being incredibly noisy. We have a lot of features, but very few data points per feature. so the model likely cant learn anything. As seen in the table in section 3 the data is very insensitive to PCA, achieving near identical or identical results for when the number of n_components is changed, likely due to the same reason as the insensitivity in the hyper parameters. Additionally, for the same reason as the heart dataset I always scaled my data.